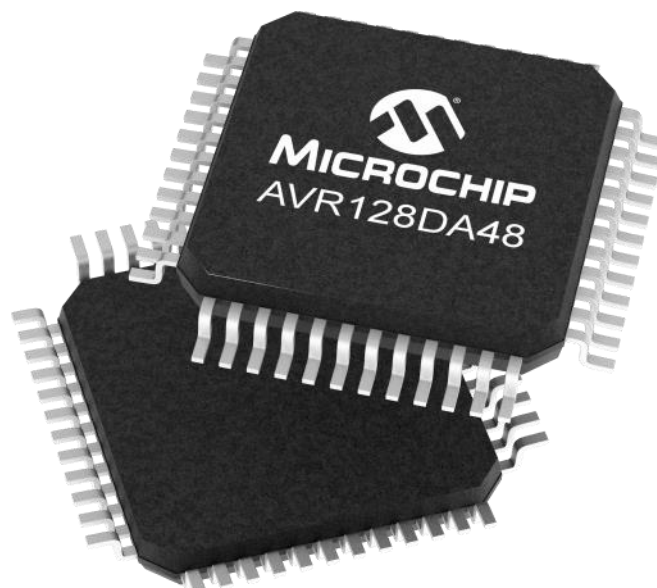
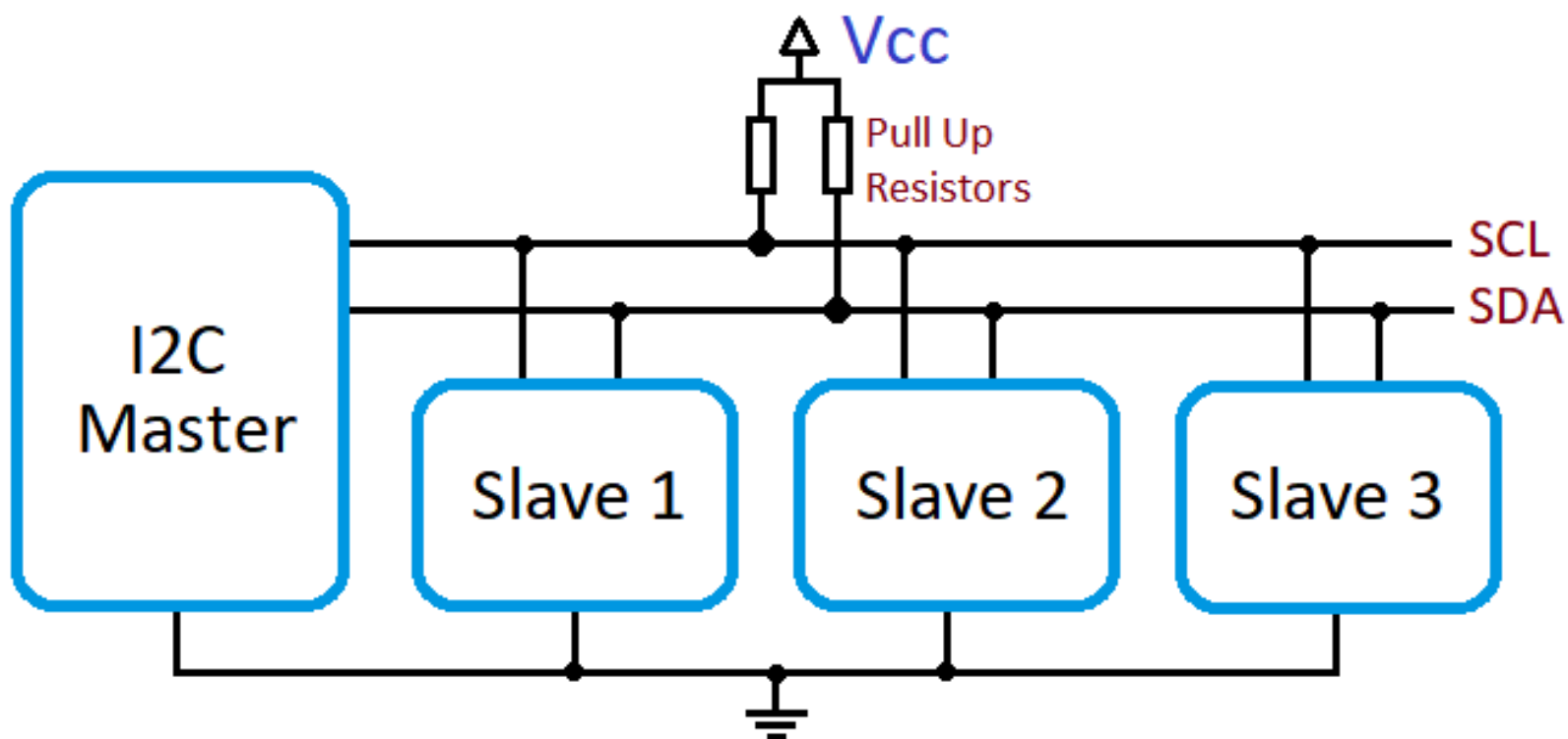


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 15 - I2C (TWI) Fundamentals



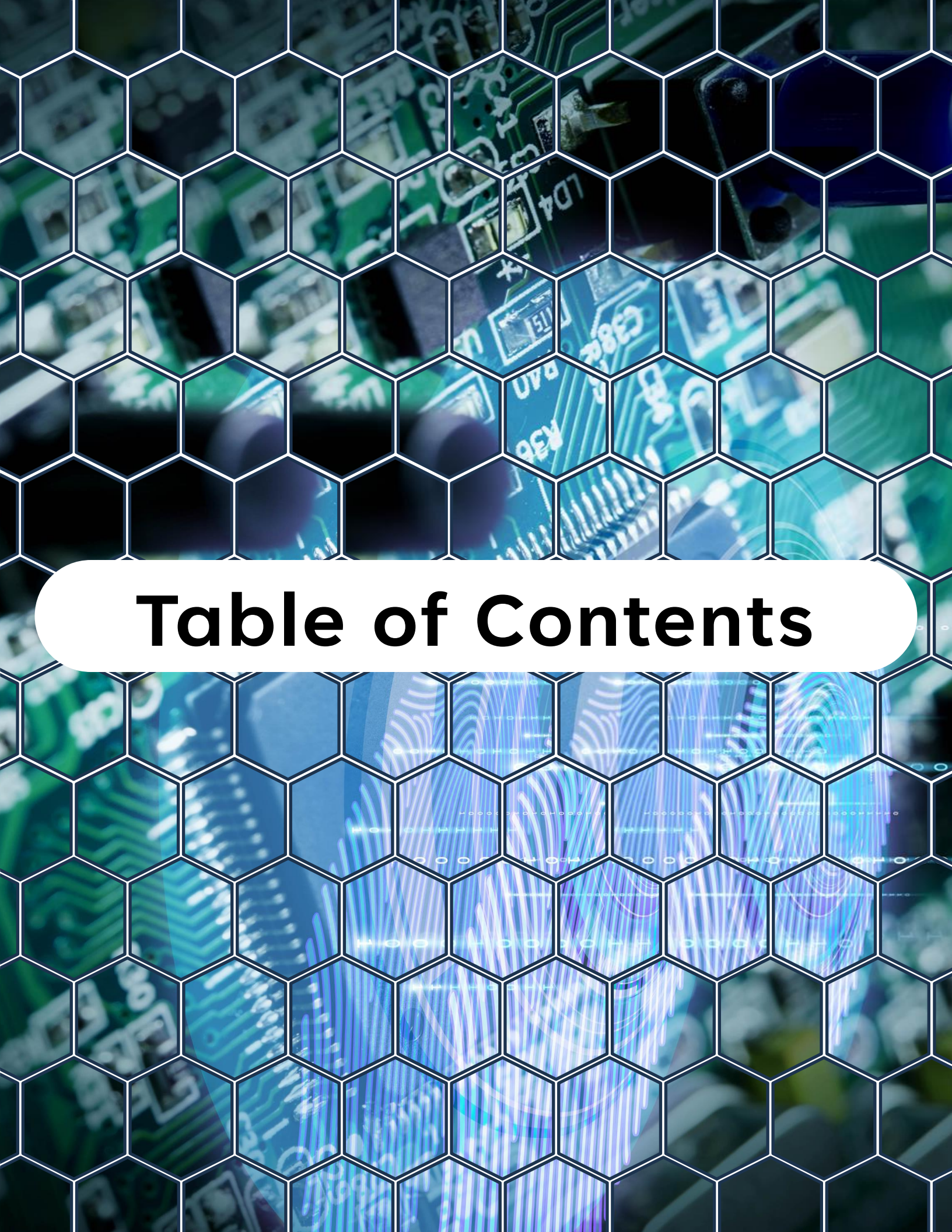
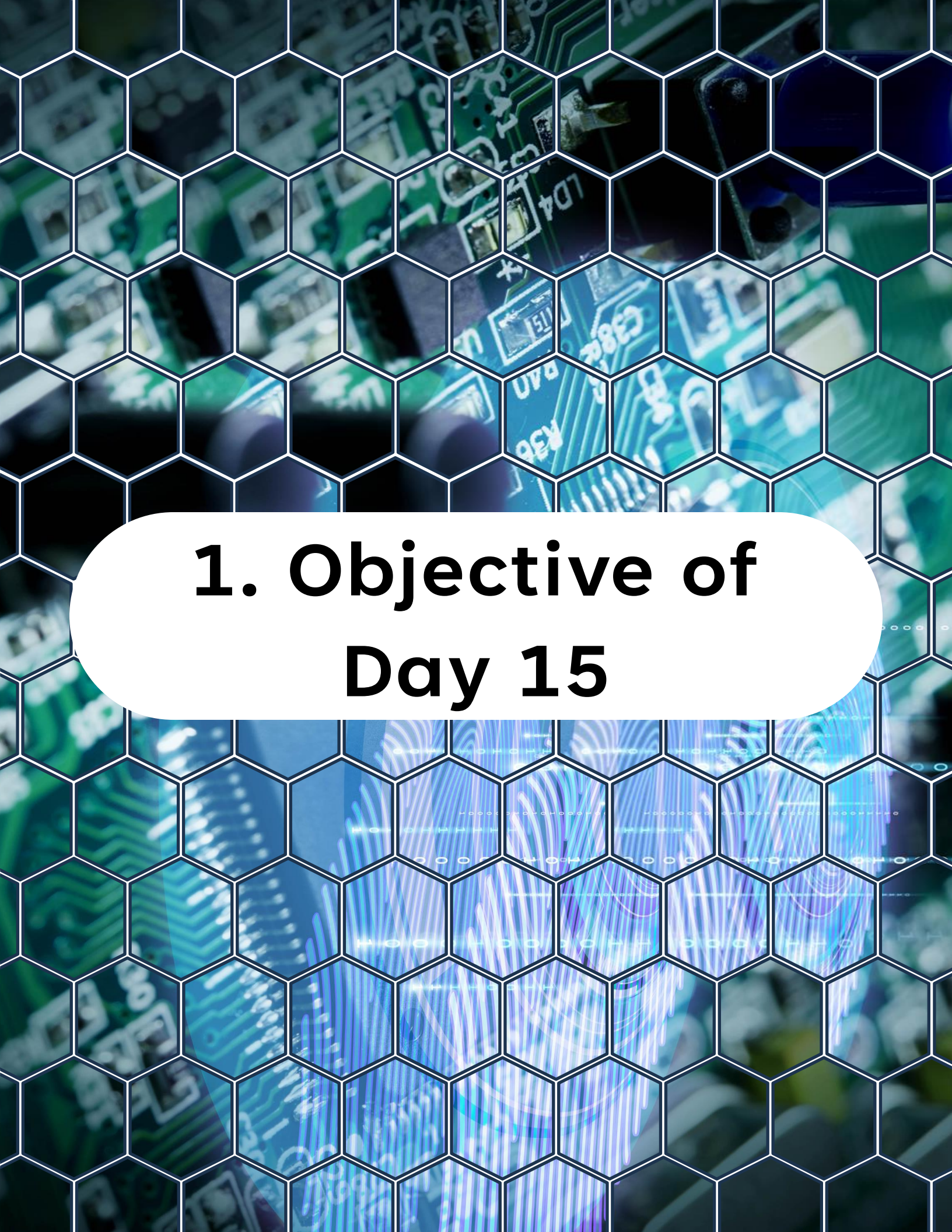


Table of Contents

Table of Contents

1. Objective of Day 15
2. What Is I2C (TWI)?
3. Electrical Fundamentals
4. Basic I2C Frame Structure
5. I2C Speeds
6. TWI Peripheral on AVR128DA48
7. Configuring TWI in Master Mode
8. Baud Rate Calculation
9. Example - Initialize TWI0 as Master (100 kHz)
10. Writing to an I2C Device (Blocking)
11. Reading from an I2C Device
12. Example - Read Register from Sensor
13. Debugging I2C
14. I2C vs SPI - Design Tradeoffs
15. What You Learned Today
16. What Comes Next



1. Objective of Day 15

1. Objective of Day 15

Today you move to a different communication philosophy.

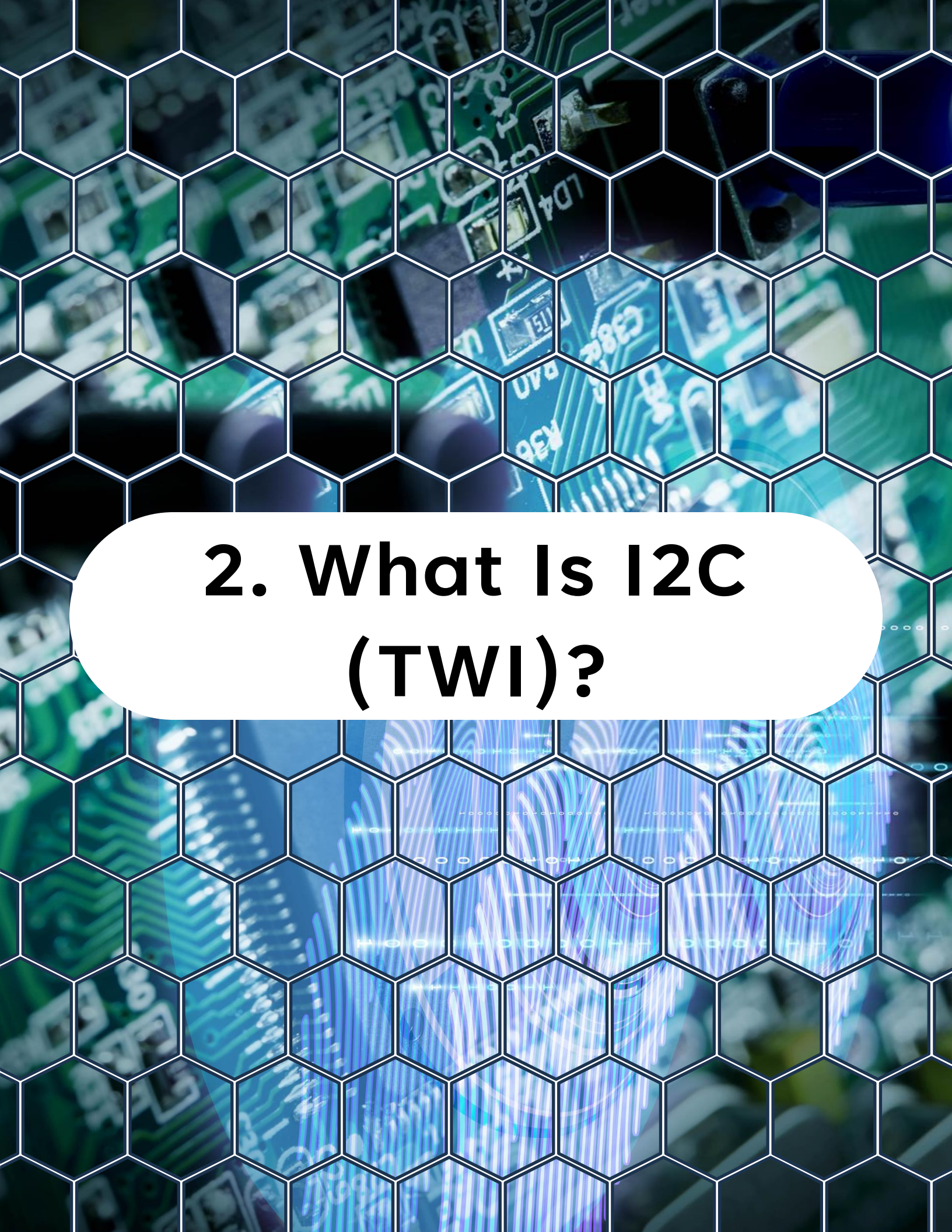
Unlike SPI, **I2C** (called **TWI** on AVR) is:

- Multi-device on two wires
- Address-based
- Open-drain with pull-ups
- Half-duplex

By the end of this day, you will:

- Understand I2C electrical fundamentals
- Configure TWI in Master mode
- Generate START and STOP conditions
- Perform basic write and read transactions
- Debug common I2C failures

This is essential because most sensors use I2C.



2. What Is I2C (TWI)?

2. What Is I2C (TWI)?

I2C stands for Inter-Integrated Circuit.

On AVR devices, it is called **TWI (Two-Wire Interface)**.

The TWI provides a simple, bidirectional, two-wire communication bus consisting of:

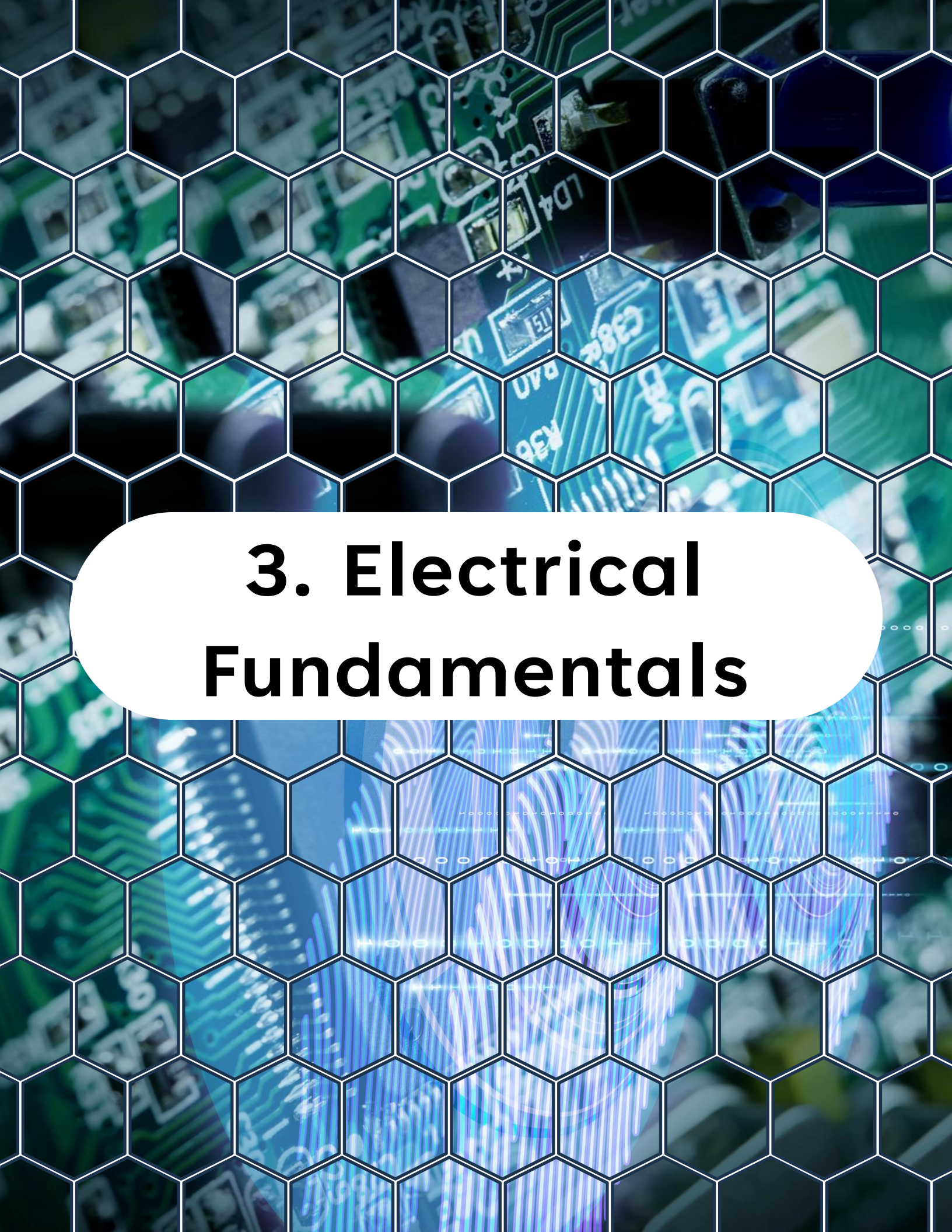
- **Serial Data Line (SDA)** for packet transfer
- **Serial Clock Line (SCL)** for the bus clock

Both lines are:

- Open-drain (or open-collector)
- Require pull-up resistors

Unlike SPI, there is:

- No chip select
- No separate MOSI/MISO
- Devices are selected by 7 or 10 bit address



3. Electrical Fundamentals

3. Electrical Fundamentals

I2C lines are not driven high.

Instead:

- Devices pull lines LOW
- Pull-up resistors pull lines HIGH

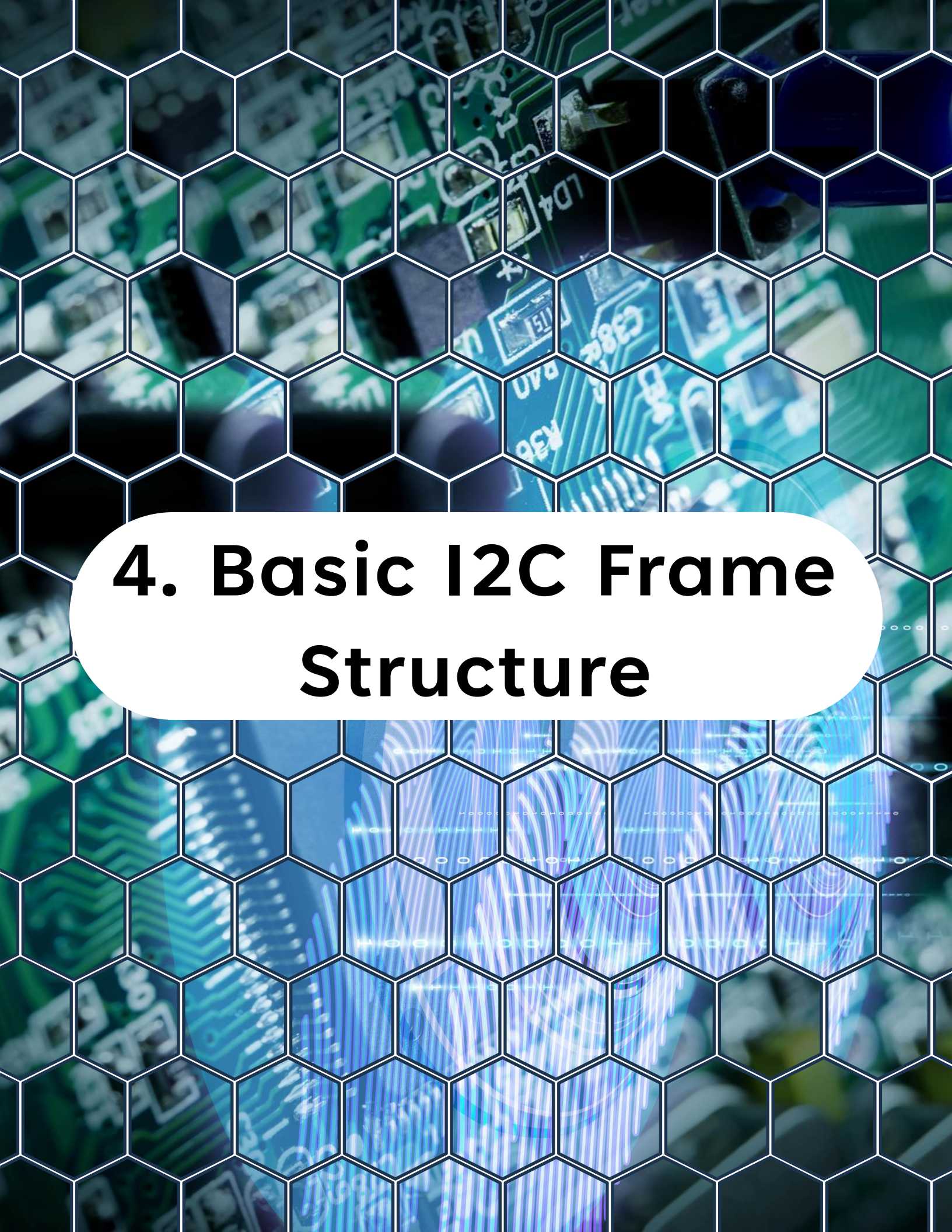
This allows:

- Multiple devices on the same bus
- Arbitration
- Clock stretching
- If you forget pull-up resistors, I2C will not work.

Typical pull-up values:

- 4.7 kOhm
- 10 kOhm

Depends on bus speed and capacitance.



4. Basic I2C Frame Structure

4. Basic I2C Frame Structure

A master transaction typically looks like:

START

ADDRESS + R/W bit

ACK

DATA

ACK

STOP

Important concepts:

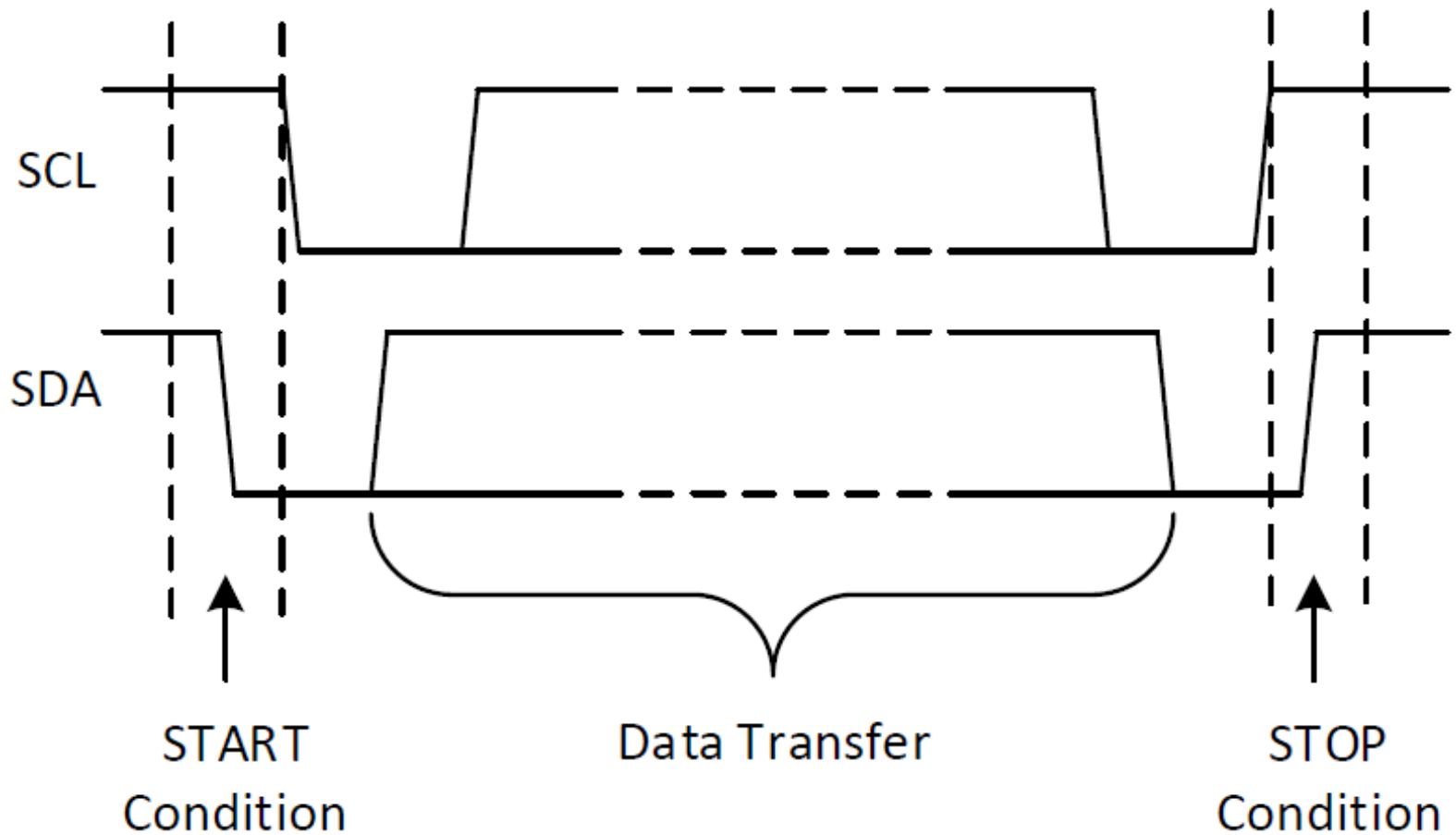
- **START:** SDA goes low while SCL high
- **STOP:** SDA goes high while SCL high
- **ACK:** Receiver pulls SDA low

If no ACK:

- Device not responding
- Wrong address
- Wiring issue

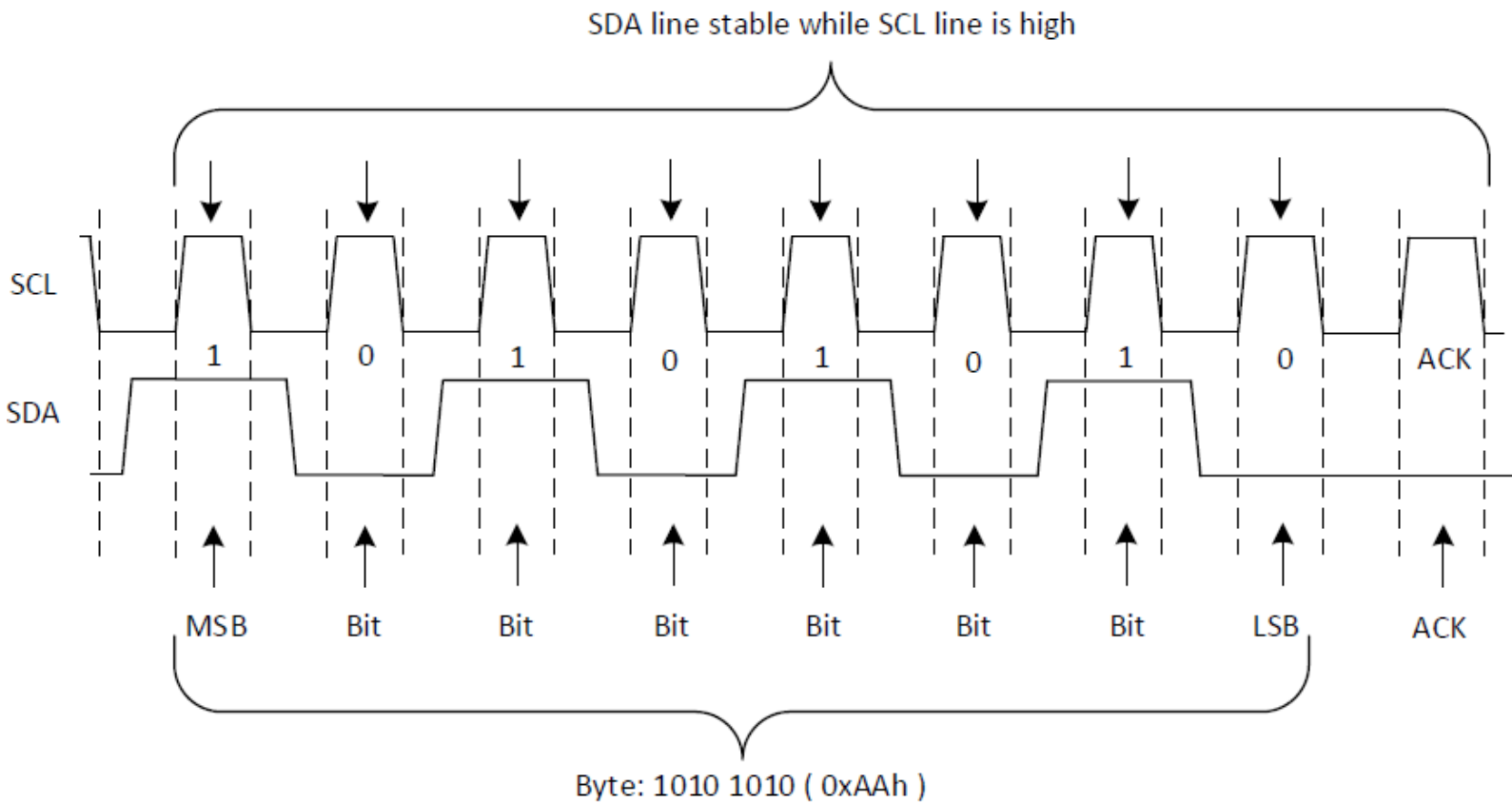
4. Basic I2C Frame Structure

Example of START and STOP Condition

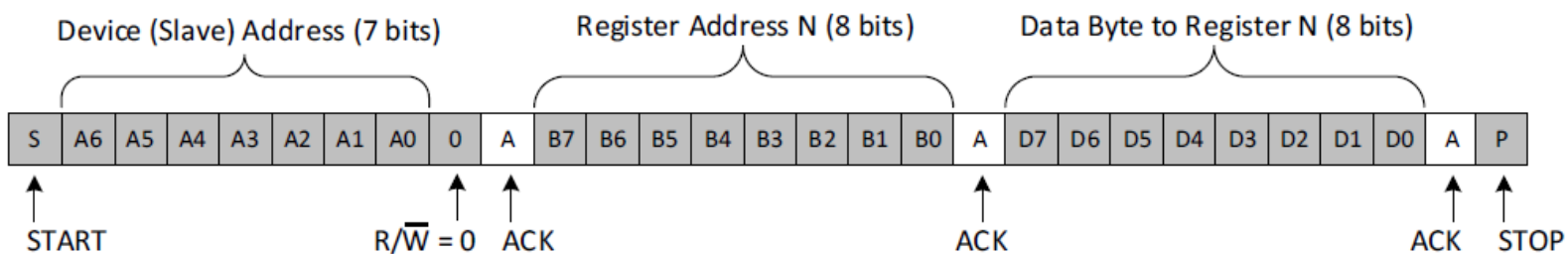


4. Basic I2C Frame Structure

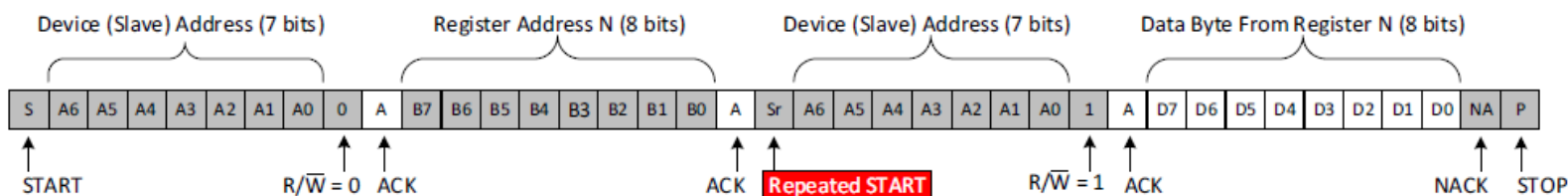
Example of Single Byte Data Transfer

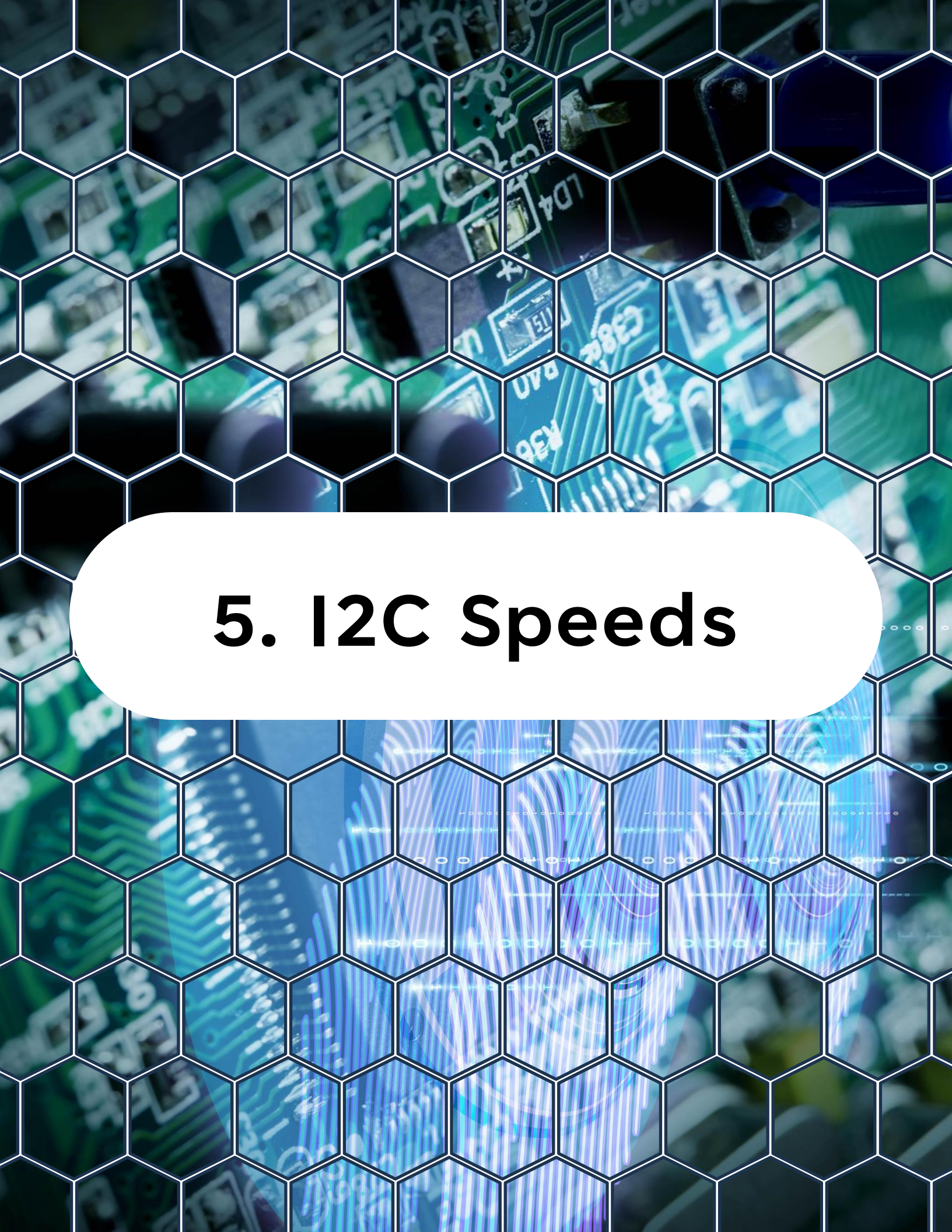


Example I2C Write to Slave Device's Register



Example I2C Read from Slave Device's Register





5. I2C Speeds

5. I2C Speeds

Common bus speeds:

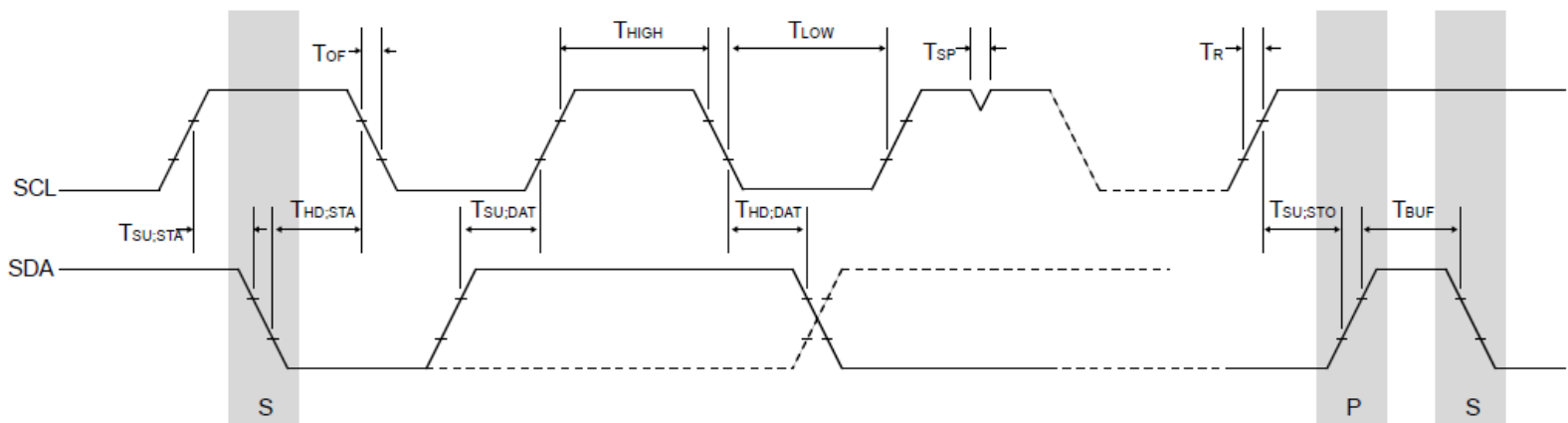
- **Standard mode** (Sm) up to 100 kHz
- **Fast mode** (Fm) up to 400 kHz
- **Fast mode Plus** (Fm+) up to 1 MHz

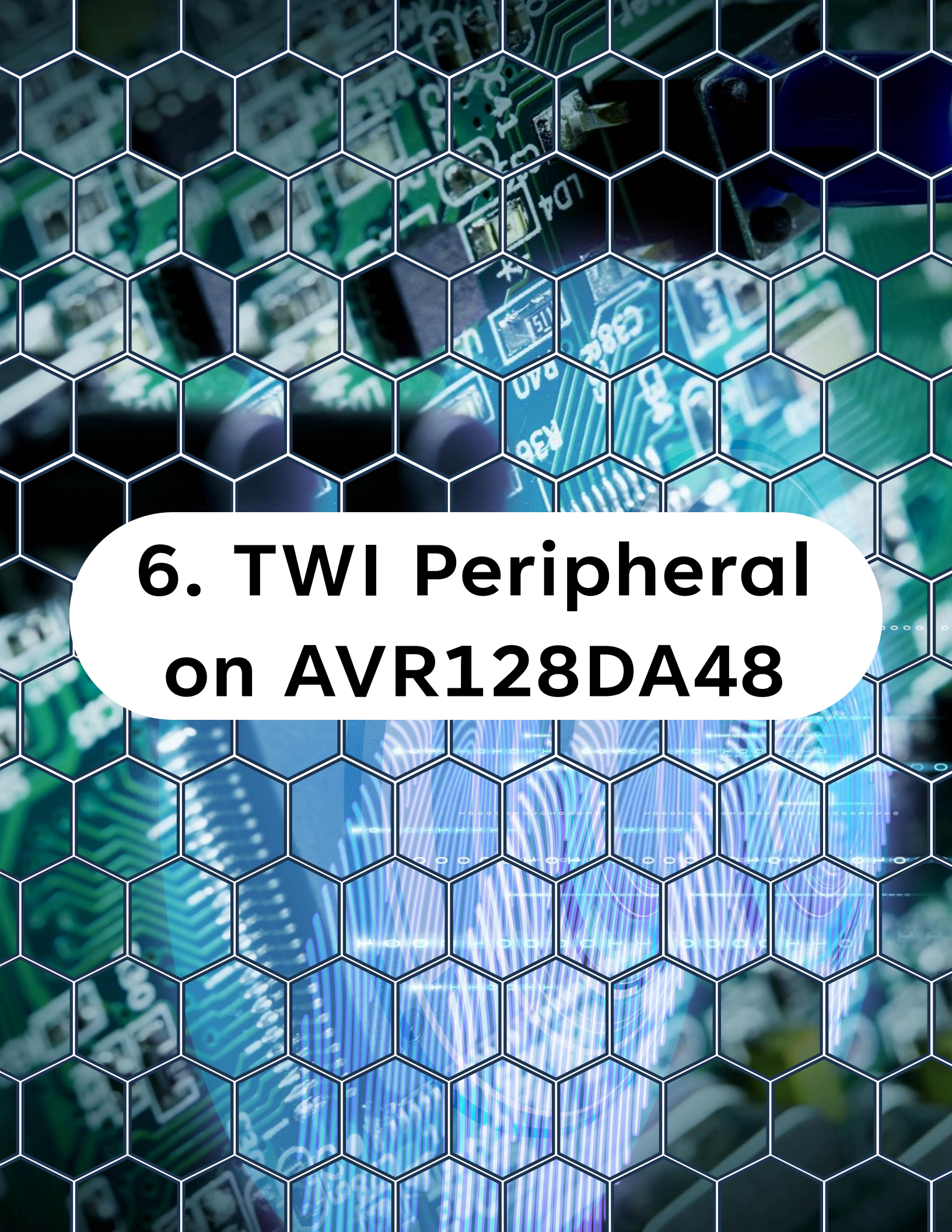
Start with 100 kHz.

Higher speed means:

- More strict timing
- Stronger pull-ups required
- Better layout discipline

SCL/SDA Timing





6. TWI Peripheral on AVR128DA48

6. TWI Peripheral on AVR128DA48

The AVR128DA48 includes TWI modules that support:

- **Master mode**
- **Slave mode**
- **Smart mode** (simplifies the application code and minimizes the user interaction needed)
- **Dual Mode** (master and the slave will operate simultaneously)
- **Quick Command Mode** (the R/W bit from the address packet denotes the command)

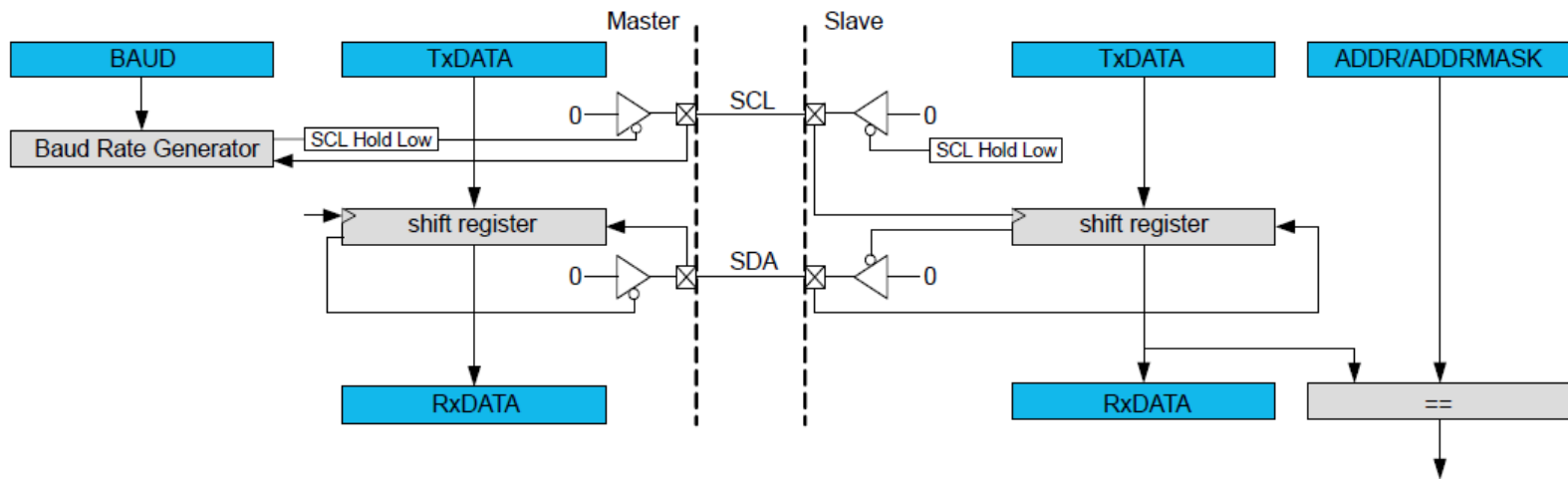
Today you focus on:

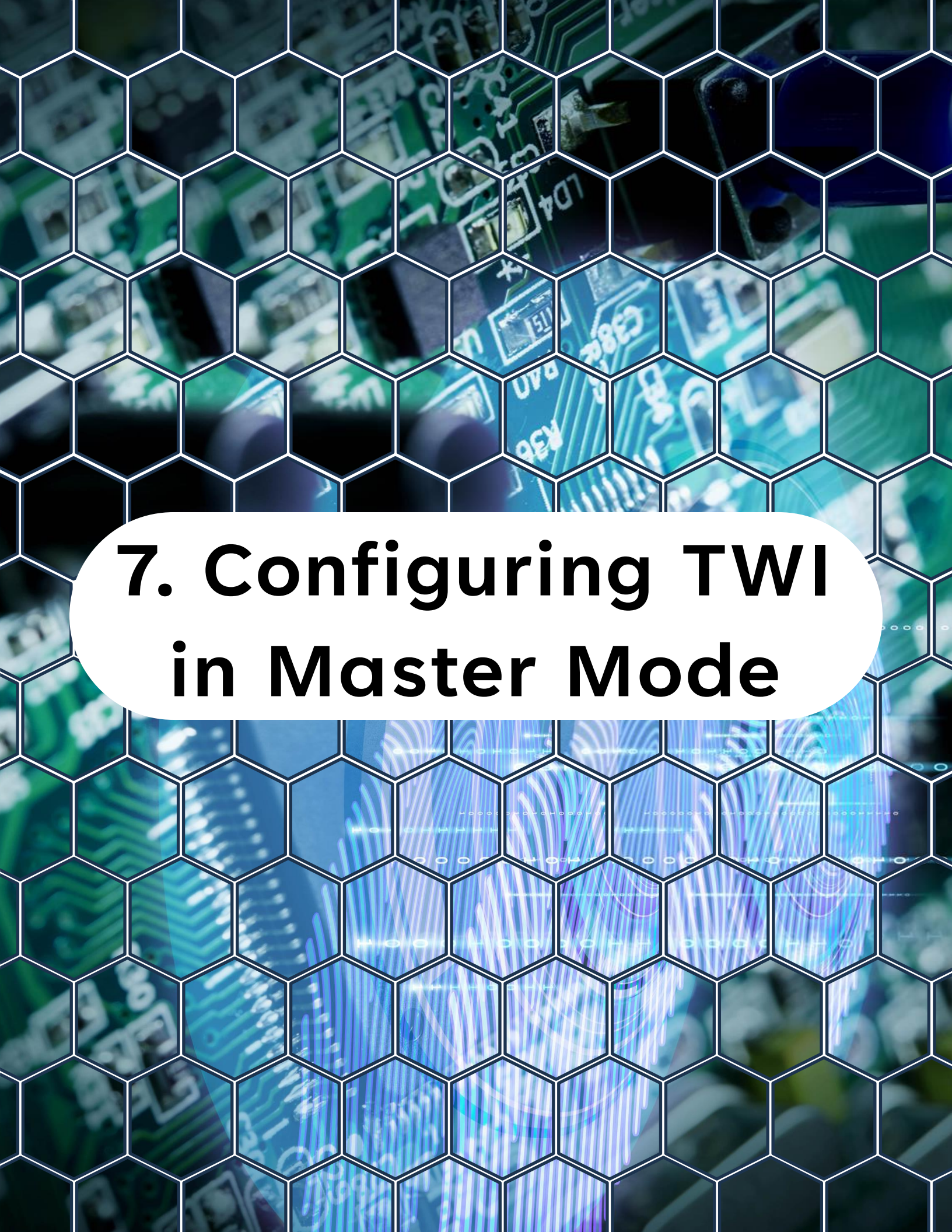
Master mode, blocking transfer.

Architecture refinement comes later.

6. TWI Peripheral on AVR128DA48

TWI Block Diagram





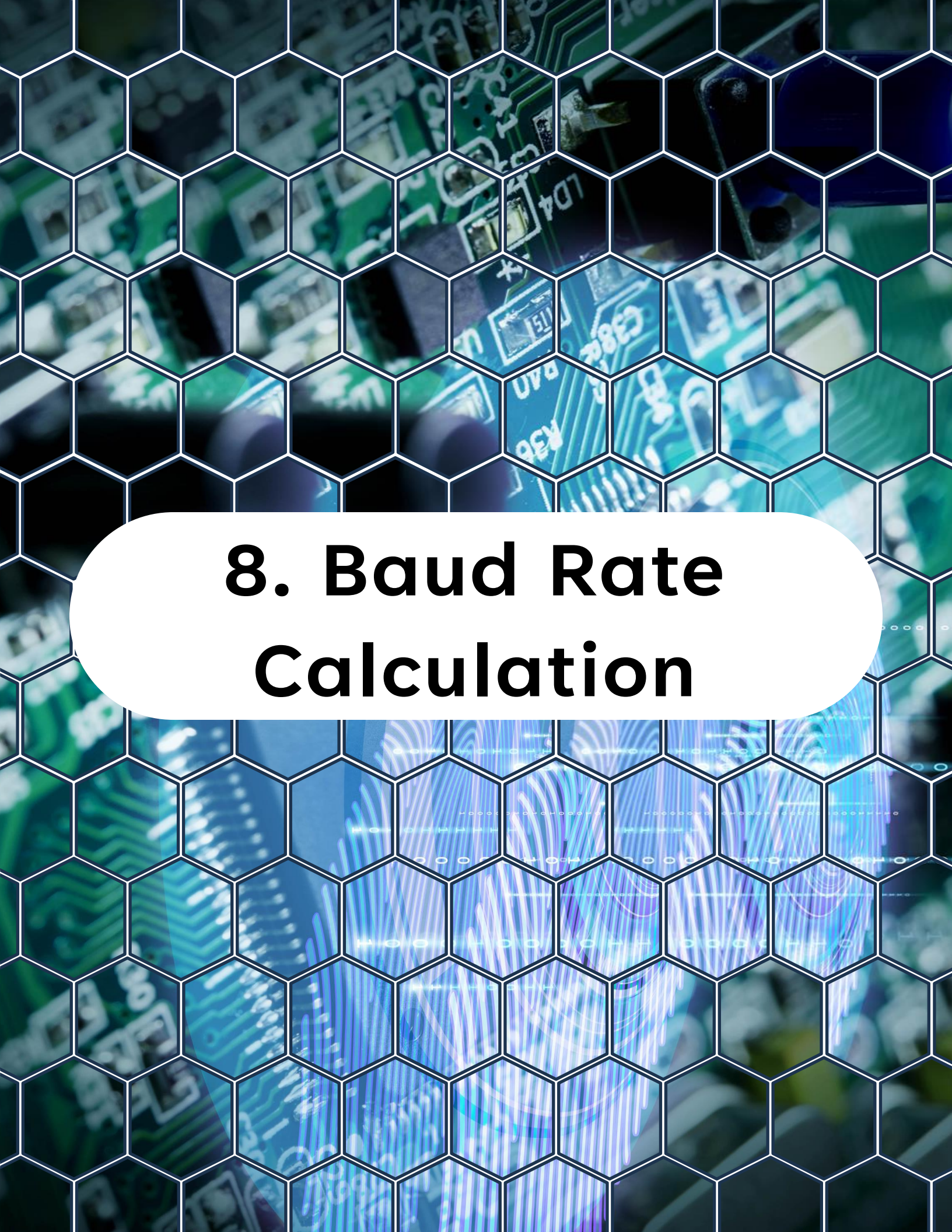
7. Configuring TWI in Master Mode

7. Configuring TWI in Master Mode

To use TWI:

1. **Configure SDA/SCL pins**
2. **Set baud rate** (The Master Baud Rate `TWIn.MBAUD` register must be written to a value that will result in a valid TWI bus clock frequency.)
3. **Enable master**
4. **Send address**
5. **Send/receive data**
6. **Send STOP**

The master starts performing a bus transaction when the Master Address (`TWIn.MADDR`) register is written with the slave address and the R/W direction bit.



8. Baud Rate Calculation

8. Baud Rate Calculation

I2C clock frequency is set by **TWI.MBAUD** register.

Simplified idea:

MBAUD depends on:

- F_CPU
- Desired I2C frequency

For 100 kHz at 24 MHz, a typical **MBAUD** value is around 75-80 (verify in datasheet formula).

Always calculate according to the device datasheet.

8. Baud Rate Calculation

Calculation of the BAUD Value

$$BAUD = \frac{f_{CLK_PER}}{2 \times f_{SCL}} - \left(5 + \frac{f_{CLK_PER} \times T_R}{2} \right)$$

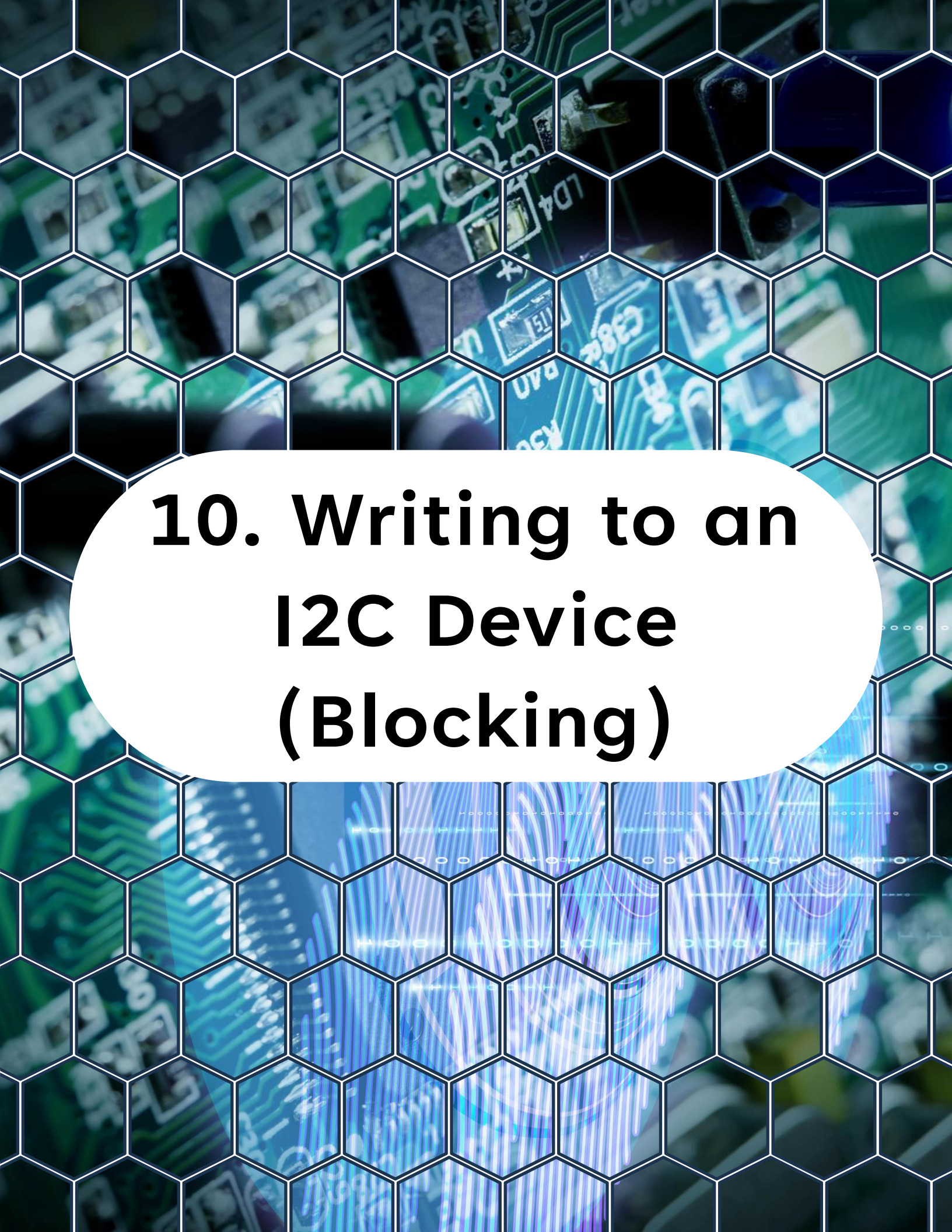
TR is determined by the bus impedance; for internal pull-ups. Refer to Electrical Characteristics for details.

**9. Example -
Initialize TWI0
as Master (100
kHz)**

9. Example - Initialize TWI0 as Master (100 kHz)

```
1  #include <avr/io.h>
2
3  #define F_CPU_HZ      (4000000UL)
4  #define I2C_FREQ_HZ (100000UL)
5  /* Rise time in nanoseconds, adjust as needed */
6  #define T_RISE        (100)
7
8  /* Example simplified MBAUD calculation (verify with datasheet) */
9
10 #define TWI_BAUD_VALUE(F_SCL, T_RISE) \
11     ((((((float)F_CPU_HZ / (float)I2C_FREQ_HZ)) - 10 - \
12     ((float)F_CPU_HZ * T_RISE / 1000000)))) / 2)
13
14
15 static void twi0_init(void)
16 {
17     /* Enable pull-ups if needed (if external not present) */
18     PORTA.PIN2CTRL |= PORT_PULLUPEN_bm; /* SDA example */
19     PORTA.PIN3CTRL |= PORT_PULLUPEN_bm; /* SCL example */
20
21     /* Set baud rate */
22     TWI0.MBAUD = (uint8_t)TWI_BAUD_VALUE;
23
24     /* Enable TWI master */
25     TWI0.MCTRLA = TWI_ENABLE_bm;
26
27     /* Set bus state to idle */
28     TWI0.MSTATUS = TWI_BUSSTATE_IDLE_gc;
29 }
```

Adjust SDA/SCL pins according to board routing.

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and surface-mount components. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center of the slide, there is a large white oval containing the title text.

10. Writing to an I2C Device (Blocking)

10. Writing to an I2C Device (Blocking)

To write:

1. Send START + address (write)
2. Send data
3. Send STOP

Example: Write one byte to device

```
1 static void twi0_write_byte(uint8_t address, uint8_t data)
2 {
3     /* Send START + address (write) */
4     TWI0.MADDR = (address << 1);
5
6     /* Wait for Write Interrupt Flag */
7     while (!(TWI0.MSTATUS & TWI_WIF_bm));
8
9     /* Check for ACK after address */
10    if (TWI0.MSTATUS & TWI_RXACK_bm)
11    {
12        /* NACK received -> send STOP and exit */
13        TWI0.MCTRLB = TWI_MCMD_STOP_gc;
14        return;
15    }
16
17    /* Send data byte */
```

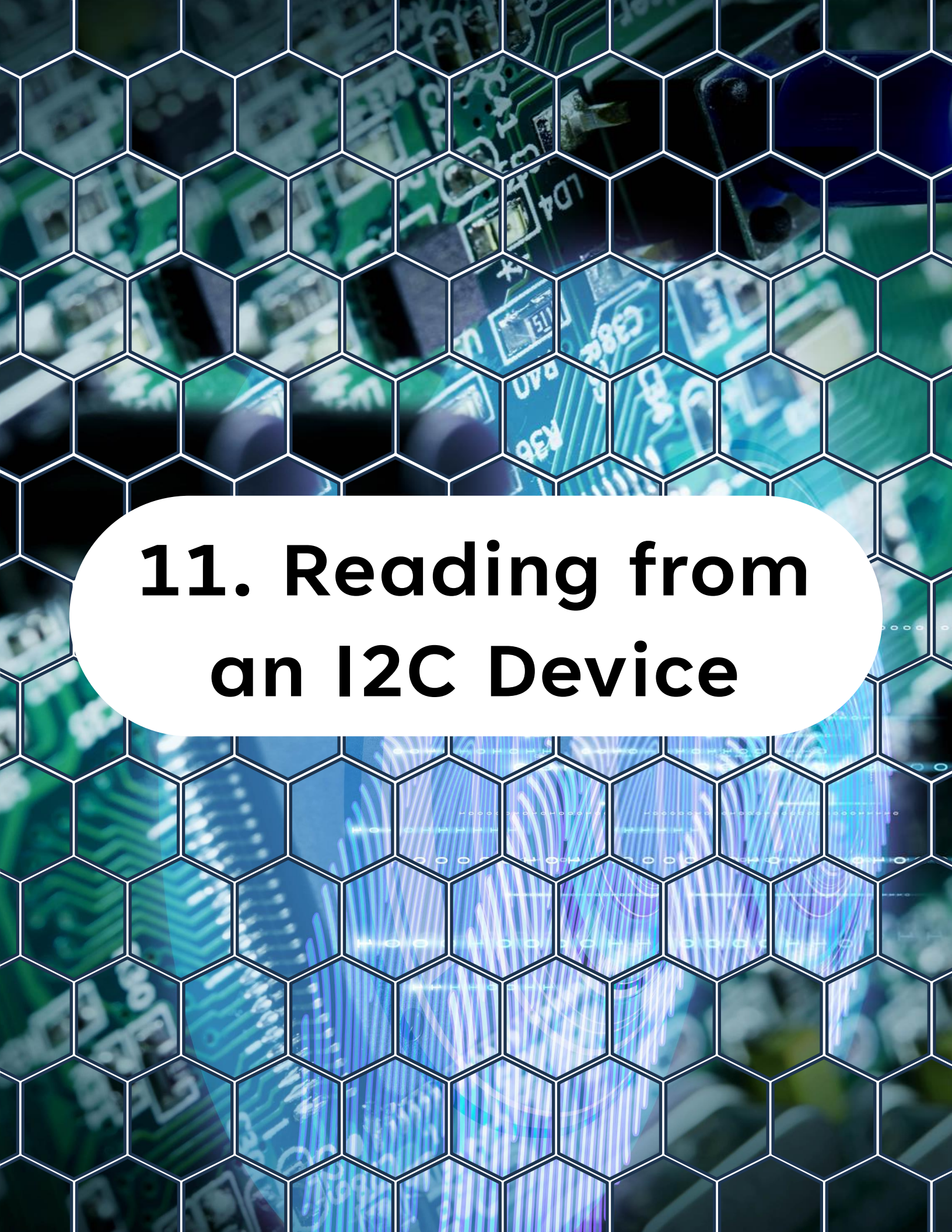
10. Writing to an I2C Device (Blocking)

```
16
17     /* Send data byte */
18     TWI0.MDATA = data;
19
20     /* Wait for Write Interrupt Flag */
21     while (!(TWI0.MSTATUS & TWI_WIF_bm));
22
23     /* Check for ACK after data */
24     if (TWI0.MSTATUS & TWI_RXACK_bm)
25     {
26         /* NACK received -> send STOP and exit */
27         TWI0.MCTRLB = TWI_MCMD_STOP_gc;
28         return;
29     }
30
31     /* Send STOP condition */
32     TWI0.MCTRLB = TWI_MCMD_STOP_gc;
33 }
```


10. Writing to an I2C Device (Blocking)

What This Code Teaches Clearly:

- **WIF** must always be checked before touching the bus again.
- **RXACK** must be checked after every transmitted byte.
- A **NACK** means the slave did not accept the byte.
- Always release the bus with **STOP** in error cases.



11. Reading from an I2C Device

11. Reading from an I2C Device

To read:

1. Send START + address (read)
2. Wait for data
3. Send NACK
4. Send STOP

Example: Read one byte



```
1 static uint8_t twi0_read_byte(uint8_t address)
2 {
3     uint8_t data = 0;
4
5     /* Send START + address (read) */
6     TWI0.MADDR = (address << 1) | 0x01;
7
8     /* Wait for Read Interrupt Flag */
9     while (!(TWI0.MSTATUS & TWI_RIF_bm));
10
11    /* Check for ACK after address */
12    if (TWI0.MSTATUS & TWI_RXACK_bm)
13    {
14        /* NACK received -> send STOP and return 0 */
```

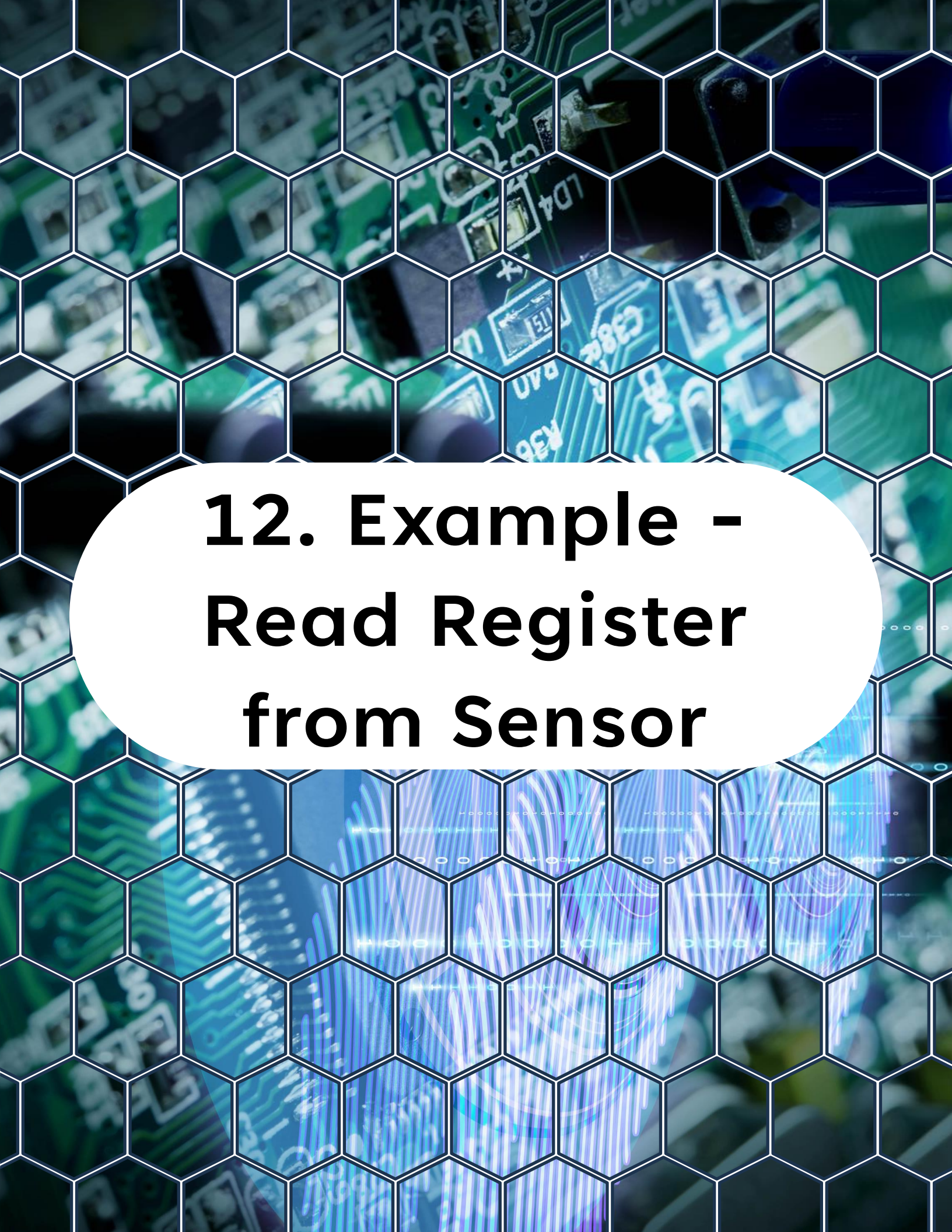

11. Reading from an I2C Device

```
10
11     /* Check for ACK after address */
12     if (TWI0.MSTATUS & TWI_RXACK_bm)
13     {
14         /* NACK received -> send STOP and return 0 */
15         TWI0.MCTRLB = TWI_MCMD_STOP_gc;
16         return 0;
17     }
18
19     /* Read received data */
20     data = TWI0.MDATA;
21
22     /* Send NACK + STOP (single byte read) */
23     TWI0.MCTRLB = TWI_ACKACT_bm | TWI_MCMD_STOP_gc;
24
25     return data;
26 }
```

11. Reading from an I2C Device

What This Example Teaches Clearly:

- Even in read mode, the master must check ACK after sending the address.
- RXACK = 1 means the slave did not respond.
- For a single byte read:
 - Master sends NACK (ACKACT = 1)
 - Then sends STOP
- Always release the bus on error.



12. Example - Read Register from Sensor

12. Example - Read Register from Sensor

Many sensors require:

- Write register address
- Repeated START
- Read data

Example pattern



```
1  uint8_t sensor_read_reg(uint8_t dev_addr, uint8_t reg)
2  {
3      uint8_t data = 0;
4
5      /* --- Step 1: Send START + device address (write) --- */
6      TWI0.MADDR = (dev_addr << 1);
7
8      while (!(TWI0.MSTATUS & TWI_WIF_bm));
9
10     /* Check ACK after address (write phase) */
11     if (TWI0.MSTATUS & TWI_RXACK_bm)
12     {
13         TWI0.MCTRLB = TWI_MCMD_STOP_gc;
14         return 0;
15     }
16
17     /* --- Step 2: Send register address --- */
18     TWI0.MDATA = reg;
19
```

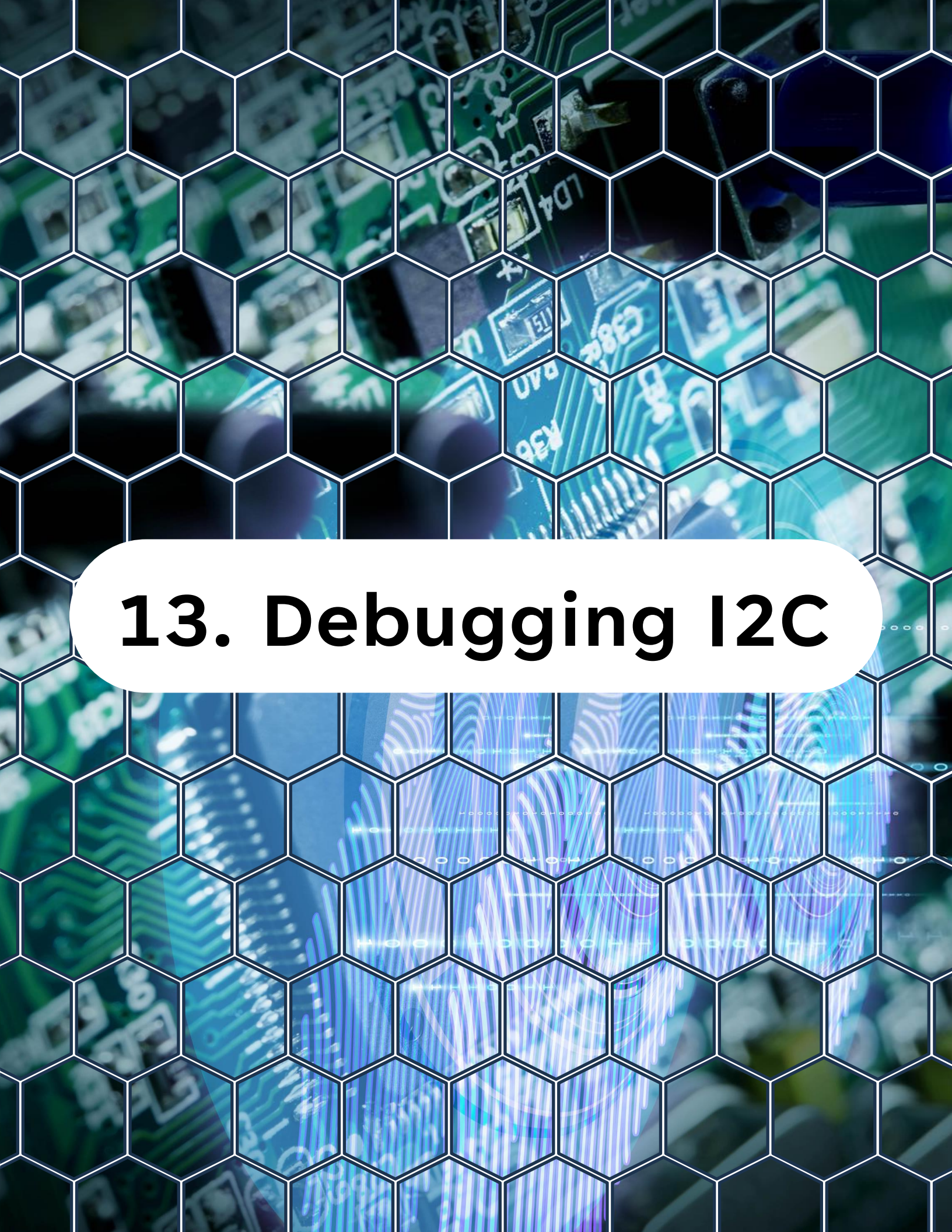
12. Example - Read Register from Sensor

```
16
17  /* --- Step 2: Send register address --- */
18  TWI0.MDATA = reg;
19
20  while (!(TWI0.MSTATUS & TWI_WIF_bm));
21
22  /* Check ACK after register byte */
23  if (TWI0.MSTATUS & TWI_RXACK_bm)
24  {
25      TWI0.MCTRLB = TWI_MCMD_STOP_gc;
26      return 0;
27  }
28
29  /* --- Step 3: Repeated START + device address (read) --- */
30  TWI0.MADDR = (dev_addr << 1) | 0x01;
31
32  while (!(TWI0.MSTATUS & TWI_RIF_bm));
33
34  /* Check ACK after address (read phase) */
35  if (TWI0.MSTATUS & TWI_RXACK_bm)
36  {
37      TWI0.MCTRLB = TWI_MCMD_STOP_gc;
38      return 0;
39  }
40
41  /* --- Step 4: Read data --- */
42  data = TWI0.MDATA;
43
44  /* Send NACK + STOP (single byte read) */
45  TWI0.MCTRLB = TWI_ACKACT_bm | TWI_MCMD_STOP_gc;
46
47  return data;
48 }
```

12. Example - Read Register from Sensor

What This Teaches Clearly:

1. **ACK** must be checked after every transmitted byte:
 - Device address (write)
 - Register byte
 - Device address (read)
2. **WIF** is used after writes.
3. **RIF** is used after read address.
4. **RXACK** is only valid after the corresponding interrupt flag is set.
5. **STOP** must be sent in all error paths.



13. Debugging I2C

13. Debugging I2C

If bus stuck LOW:

- Missing pull-ups
- Device holding line low
- Clock stretching issue

If no ACK:

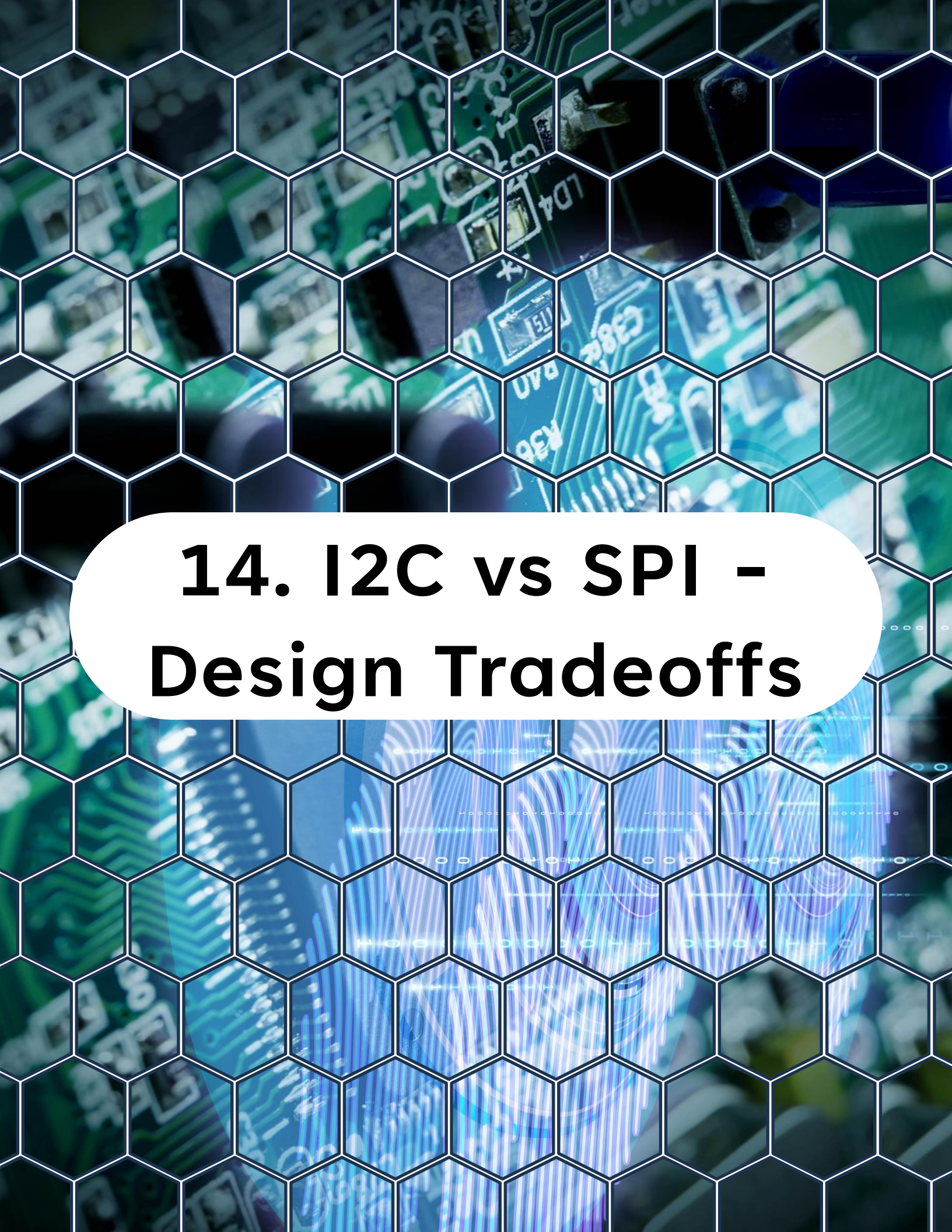
- Wrong address
- Wrong wiring
- Device not powered

If random failures:

- Weak pull-ups
- Too high speed
- Noise on lines

Use logic analyzer whenever possible.

I2C debugging is electrical and logical.

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and surface-mount components. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white rounded rectangle contains the title text in bold black font.

14. I2C vs SPI - Design Tradeoffs

14. I2C vs SPI - Design Tradeoffs

SPI:

- Faster
- More wires
- Simpler protocol
- Dedicated chip select per device

I2C:

- Fewer wires
- Address-based
- Slower
- Easier multi-device wiring

Many systems use both.



15. What You Learned Today

15. What You Learned Today

By the end of Day 15, you now understand:

- I2C electrical model (open-drain + pull-ups)
- START, STOP, ACK behavior
- TWI master initialization
- Basic write and read transactions
- Register read pattern used by sensors
- How to debug common I2C problems

Your MCU can now talk to almost any sensor on the market.



16. What Comes Next

16. What Comes Next

Day 16 - I2C Architecture and Interrupt-Driven TWI

Next you will:

- Build a reusable TWI driver
- Implement non-blocking I2C transfers
- Handle multi-byte reads properly
- Add timeout and error handling
- Prepare for real sensor drivers

This is where I2C becomes production-ready instead of demo-level.

You can find all source code, examples, and updates for this course in the GitHub repository, make sure to visit it, explore the files, and clone it so you can follow along hands-on.

<https://github.com/god233012yamil/Bare-Metal-Embedded-Systems-with-AVR128DA48-Course>